

Implementation and Verification of a CAN-FD Bus Transceiver in VHDL

Mads Richardt (s224948)

February 28, 2026

Contents

0.1	Design Space Exploration	1
0.1.1	Company Design Constraints	2
0.1.2	Monolithic vs. Layered Architecture	2
0.1.3	Combined vs. Separated TX/RX FSMs	3
0.1.4	Per-Field vs. Per-Phase FSM Granularity	4
0.1.5	Interface Record Design	4
0.1.6	Dual CRC Data Feeds	4

0.1 Design Space Exploration

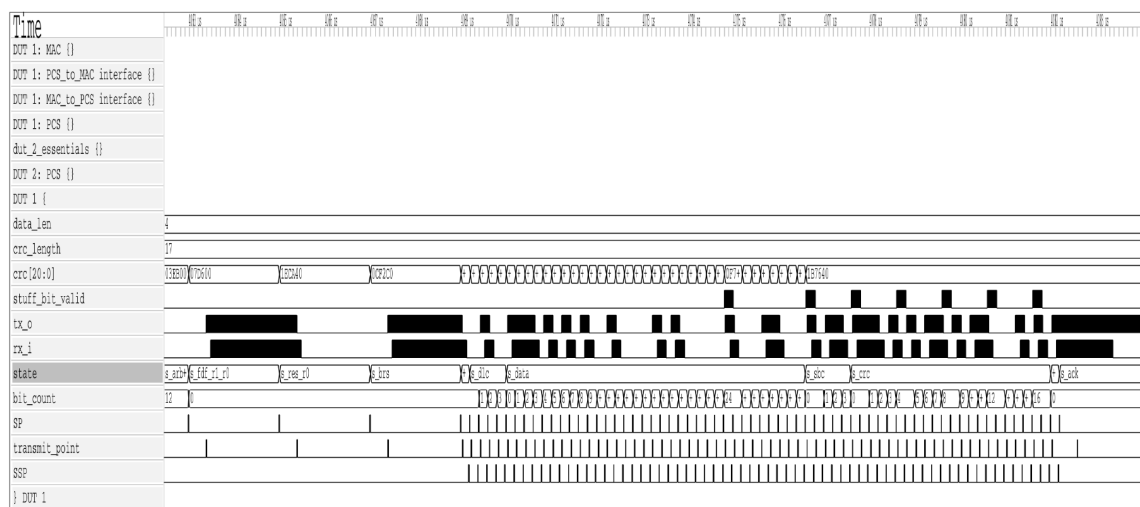


Figure 1: GTKWave waveform of a CAN FD frame transmission captured from the `can_mac_pcs_fce_tb` testbench, showing MAC state, bit count, TX/RX bus, sample points, and CRC progression.

Before settling on the final architecture, several design alternatives were evaluated. The exploration drew on three sources: the existing in-house CAN Classic controller (Section ??), the open-source CTU CAN FD core [1], [2], and the ISO 11898-1 standard’s own layered reference model [3]. This section documents the key design decisions, the alternatives that were considered, and the constraints that shaped the outcome.

0.1.1 Company Design Constraints

The company imposes a set of non-negotiable constraints on all FPGA IP modules. These constraints were established before the design exploration began and are not subject to trade-off analysis - they define the feasible design space.

std_logic-only entity ports. All module interfaces must use `std_logic` and `std_logic_vector` exclusively. No custom enumeration types, booleans, or integers are permitted on entity ports. This mandate ensures that every module can be integrated into the company's existing synthesis and static analysis toolchain without adapter logic. The constraint precludes designs that expose protocol-level enumerations (such as FSM state types or frame format selectors) on their interfaces - a pattern used in the existing in-house controller, where `t_fsm_can_state` and `t_node_state` enums appear on debug ports. Internally, modules may use any VHDL-2008 type; the restriction applies only at module boundaries.

Per-module directory structure. Each module resides in its own directory with `hdl_src/`, `hdl_tb/`, and `test_case/` subdirectories. This convention is enforced across all company FPGA projects and must be followed by any new IP. It rules out monolithic file organizations where multiple entities share a single source directory.

Avalon-ST streaming interfaces. Data-transfer interfaces between modules use the Avalon-ST protocol (data, valid, ready, sop, eop). The existing FPGA infrastructure relies on this protocol for inter-module communication, and the CAN transceiver must integrate natively. This constraint excludes IP cores with AXI, APB, or custom register-map interfaces (such as CTU CAN FD's Avalon-MM or the Xilinx core's AXI4-Lite) without an adaptation layer.

Shared gen_crc IP block. The company maintains a reusable, parameterized CRC generator (`gen_crc`) in its IP library. All CRC computation must instantiate this block rather than implementing polynomial division inline. The CRC wrapper module must therefore be a structural shell that instantiates `gen_crc` with the appropriate polynomial, initial value, and data width for each CRC variant.

0.1.2 Monolithic vs. Layered Architecture

The most fundamental architectural decision was whether to extend the existing monolithic controller or adopt a layered decomposition.

The existing controller uses a flat architecture: a single FSM (`can_fsm`, 810 lines) directly drives the bus output, reads the bus input, manages bit counting, handles stuff bit insertion, coordinates CRC computation, and updates error counters - all in one clocked process. This approach minimizes inter-module latency and signal fanout, since every decision is made in a single combinational cone. For CAN Classic, where the frame has at most two format variants (base and extended) and a single bit rate, the complexity is manageable.

CAN FD, however, introduces four frame formats, dual bit rates, fixed bit stuffing with SBC encoding, three CRC polynomials, and a richer error model. Extending the monolithic FSM to cover these features would require nested conditionals on the format type in nearly every state, increasing the cyclomatic complexity well beyond what is practical to verify or maintain. The existing controller's `s_arbitration` state already contains 70 lines of interleaved TX/RX logic for two frame formats; scaling this to four formats with additional control fields (BRS, ESI, SBC) would roughly triple the state's complexity.

The alternative - and the approach adopted - is to follow the ISO 11898-1 reference model, which decomposes the data link layer into LLC, MAC, and PCS sub-layers with a separate FCE. Each sub-layer has a

well-defined service interface (described in Section ??), and each can be implemented and verified independently. This decomposition introduces inter-module interfaces and pipeline latency, but it confines format-specific complexity to the module where it belongs: the MAC FSM handles frame field sequencing, the bit stuffer handles stuff bit insertion and SBC generation, and the CRC engine handles polynomial computation. No module needs to know about all three concerns simultaneously.

CTU CAN FD [1] takes a middle path: it separates protocol processing (in a CAN Core block) from buffer management and register access, but the protocol core itself remains a large monolithic unit. This is a reasonable trade-off for a general-purpose IP core that must support message filtering, multiple TX buffers, and DMA - features that require tight coupling between the protocol engine and the buffer subsystem. For the narrower scope of this project (protocol engine only, no message RAM or filtering), the full ISO layered decomposition is feasible and preferred because it directly maps each module to a testable subset of the standard's requirements.

0.1.3 Combined vs. Separated TX/RX FSMs

The existing controller uses a single FSM for both transmission and reception, switching between roles via an `is_transmitter` flag. Every state contains parallel `if transmit_i` and `elsif sample_rx_i` branches, one for the transmitter path and one for the receiver path. This approach has two advantages: it avoids duplicating shared state (bit counters, format detection) and it naturally handles the transmitter-to-receiver role switch that occurs when a node loses arbitration.

The disadvantage is that TX and RX logic evolve together. Adding a TX-only feature (such as the data-phase bit rate switch) requires modifying states that also contain RX logic, and vice versa. In CAN FD, the TX and RX paths diverge more than in CAN Classic: the transmitter drives BRS and ESI based on local state, while the receiver samples and validates them. The transmitter feeds the CRC engine with transmitted bits, while the receiver feeds it with received bits (including dynamic stuff bits in the arbitration region, per ISO 6.6.13.3). Encoding both paths in every state produces deeply nested conditionals that are difficult to review and difficult to cover in verification.

The chosen design uses separate TX and RX FSMs (`can_mac_fsm_tx` with 19 states, `can_mac_fsm_rx` with 17 states), each with its own bit stuffer and CRC interface. The two FSMs share no state. Arbitration loss in the TX FSM triggers a `transfer_status` change, but the actual reception of the frame is handled by the RX FSM operating independently on the same bus. This separation means that the TX FSM can be verified exhaustively against TX-side requirements without any RX stimulus beyond bus-level loopback, and conversely for the RX FSM.

The cost of separation is that sub-components used by both paths - the bit stuffer (`can_mac_bs`) and CRC engine (`can_mac_crc`) - must be either shared with multiplexing logic or duplicated. The chosen design duplicates both: `can_mac_tx` instantiates its own `can_mac_bs` and `can_mac_crc`, and `can_mac_rx` does the same. This trades a modest increase in area for complete independence between the two paths - each FSM drives its bit stuffer and CRC engine through its own interface signals without any arbitration or routing logic. The duplication is justified because the bit stuffer and CRC engine are small combinational-plus-register modules (under 120 and 80 lines respectively), while a shared-resource approach would require multiplexing logic in the `can_mac` wrapper and careful coordination to prevent one path's state from corrupting the other's. The `can_mac` wrapper (Section ??) therefore contains no datapath logic at all - it is purely structural, instantiating `can_mac_tx`, `can_mac_rx`, and `can_fce`, and merging only the FCE error signals.

0.1.4 Per-Field vs. Per-Phase FSM Granularity

A second FSM design decision was the granularity of states. The existing controller uses per-phase states: `s_arbitration` covers all ID bits, SRR, IDE, and RTR; `s_control` covers reserved bits and DLC; `s_data` covers all data bytes. Within each state, a bit counter and conditional logic dispatch the correct action for each bit position.

This per-phase approach keeps the state count low (18 states in the existing controller) but pushes complexity into the bit-counting logic. The `s_arbitration` state must distinguish between base and extended formats using the bit counter, handle the SRR/IDE branching point at bit 12/13, and detect arbitration loss - all in a single state with a single counter.

The alternative is per-field states, where each protocol field (SOF, ID, SRR/RRS, IDE, FDF, RES, BRS, ESI, DLC, Data, SBC, CRC, ACK, EOF) has its own state. This increases the state count (19 TX states, 17 RX states) but eliminates most bit-counter conditionals: when the FSM is in `s_brs`, it knows exactly which bit it is processing without consulting a counter. Format-dependent transitions are handled by the state graph itself - for example, the FSM transitions from `s_ide` to `s_fdf_r1_r0` for all formats, but from `s_fdf_r1_r0` it may go to `s_dlc` (Classic) or `s_res_r0` then `s_brs` (FD). Each state contains only the logic relevant to its specific field, and named guard predicates (`v_in_arbitration_field`, `v_in_dynamic_stuff_field`, `v_in_fixed_stuff_field`) replace repeated bit-range comparisons.

The per-field approach was chosen because it aligns with the verification plan: each field in the frame structure corresponds to a set of requirements in the verification plan, and each FSM state can be traced directly to those requirements. It also simplifies formal verification, since PSL assertions can reference state names rather than counter ranges.

0.1.5 Interface Record Design

The company's `std_logic`-only port constraint does not preclude the use of VHDL record types on entity ports - records of `std_logic` and `std_logic_vector` fields satisfy the constraint. The design uses typed record interfaces (e.g., `t_can_mac_pcs_if_m2s`, `t_can_mac_fsm_bs_if_s2m`) to bundle related signals into a single port. Each record type has a corresponding reset constant (e.g., `c_can_mac_pcs_if_m2s_reset`), ensuring that every module can be reset to a known state without manually enumerating each field.

The alternative - flat port lists with individual `std_logic` signals - was used in the existing controller, where the FSM entity has 20 individual ports for its various sub-module interfaces. This approach becomes unwieldy as the number of inter-module signals grows: the new design has over 60 inter-module signals across its interfaces, and bundling them into records reduces the port list from dozens of lines to six record-typed ports per FSM entity.

The naming convention follows the data-flow direction: `m2s/s2m` (master-to-slave/slave-to-master) for control interfaces, and `s2d/d2s` (source-to-destination/destination-to-source) for Avalon-ST data-transfer interfaces. This convention is consistent with the company's existing interface naming and makes the direction of data flow explicit at every port.

0.1.6 Dual CRC Data Feeds

A non-obvious design decision concerns the CRC engine's data input. In CAN Classic, the CRC is computed over the raw bit stream excluding stuff bits. In CAN FD, the CRC is computed over the raw bit stream in

most of the frame, but in the arbitration region the computation includes dynamic stuff bits - a difference from Classic that exists because the FD CRC must protect the stuff bit count as well as the data. This means that a single data feed to the CRC engine is insufficient: the Classic CRC needs the de-stuffed stream, while the FD CRC needs the stuffed stream during arbitration and the de-stuffed stream elsewhere.

The chosen solution exposes two data inputs on the CRC interface: `data_cc` (Classic CAN data, always de-stuffed) and `data_fd` (FD data, which includes dynamic stuff bits during the arbitration region). The FSM drives both feeds, and the CRC wrapper routes `data_cc` to the CRC-15 engine and `data_fd` to the CRC-17 and CRC-21 engines. This avoids multiplexing logic inside the CRC module and keeps the CRC wrapper purely structural - it instantiates three `gen_crc` blocks and an output mux, with no protocol knowledge.

- [1] M. Jeřábek and P. Píša, “CTU CAN FD — open-source CAN FD IP core.” 2019. Available: https://gitlab.fel.cvut.cz/canbus/ctucanfd_ip_core
- [2] M. Jeřábek, “Open-source and open-hardware CAN FD protocol support,” PhD thesis, Czech Technical University in Prague, 2019. Available: <https://dspace.cvut.cz/handle/10467/82630>
- [3] International Organization for Standardization, *Road vehicles — controller area network (CAN) — Part 1: Data link layer and physical coding sublayer*. 2024.